Component A:

Interview Notes

"When [people] needs to [task], they currently [workaround], which causes [pain]."

All purchases across all classes, when you order, when you arrive, good condition or needs to be returned across a lot of different vendors

using excel sheets

easier way for students to submit, review, and approved by instructors

high volume: 20 - 150 orders a month for 3 months

Variable budget across all classes

need to save receipt upon arrival and packing slips (physical and scanned) until verified in workday (3 days - 2 weeks)

used Google Forms, however receive a lot of incomplete or wrong information forms. hard to mitigate mistakes

- Frustrating have to look through all the data but is incomplete or blank

Form: Team #, CFO, provider/supplier, quantity, name of item, price, link to purchase, notes, approval status from instructor

- Non-amazon doesnt let user tracks who ordered what
    - lock with non-amazon features
- Amazon can generate a report

Generate a consolidate result, everything is manual right now
Approvals are done once a week

Submission process easier for students, is there a way for students to interact with Dorothy to also want to contribute to

- Students give all the information to Dorothy

Excel sheet that pulls from all other tabs

- Totals from tabs 1-4 comes into tab 5
    - auto sum the total sum

Have to double check item arrived
Amazon: returns - have to track refunds
Students are not fully aware of the entire process, have CFO to have a point of contact in a group

- Students might not be proactive, and wait until the final solution

Dorothy's job to make sure if items are on back order and if students are okay with waiting

Priority List:

- Better solution on student submitting order forms for purchasing (rn editing excel sheets), have to lock after deadline time, every week purchase
    - if colors/notes on excel reach out to CFO, if no color, can assume purchased

Want everything to automate into a system but still be able to have control and manipulate it

different spreadsheet for each class with purchasing (usually 2)

for launch need it for the whole duration

- never delete it, have an archive

Okay with replacing entirely

- manually excels
- Would want something that can be exported into csv format

Provider is a dropdown, CFO can be made a dropdown (first and last name)

Work with Team number format

- Google form into CSV that goes into google sheet

CFO has to be the one to submit the purchase requests

- Keep this ability

Question

When Dorothy and CFOs need to submit, track, and approve purchasing orders across multiple classes and vendors, they currently manually manage Excel sheets, which causes incomplete and incorrect submissions, tedious manual consolidation, poor order status visibility, and an overwhelming administrative burden managing 20–150 orders a month.

Component B:

AI Usage Log: Document at least 3 interactions with AI tools (include Cursor prompts from build/iteration). For each, write:

1.

Prompt

Build a Streamlit app called ProcureGIX — a lightweight internal procurement tool for a university program coordinator (Dorothy) who manages student purchasing orders across multiple classes.

The Core Problem

Students currently give all their order information directly to Dorothy, who manually enters everything into Excel sheets. The goal is a simple submission form that students can fill out themselves — reducing Dorothy's data entry burden while keeping her in control of the final consolidated view.

Users

Students (CFO role) — submit their own order requests through a clean form

Dorothy (admin) — reviews all submissions in one consolidated table, manages status, exports data

Core Features

Student Submission Form

Fields: Team #, CFO Name (dropdown), Provider/Supplier (dropdown), Item Name, Quantity, Price, Link to Purchase, Notes

All fields required — no incomplete submissions allowed

Class selector (Class 1 or Class 2)

Simple and fast to fill out — students should be able to submit in under 2 minutes

Dorothy's Admin Dashboard

Consolidated table pulling from all class submissions into one master view

Filter by class, team, status, provider

Per-class subtotals and a master grand total

Export to CSV at any time

Nothing is ever deleted — archive only

Tech Requirements

Streamlit frontend

SQLite for local data persistence

Simple role selector at top (Student vs Admin) — no auth needed for MVP

Clean minimal UI using native Streamlit components

Result

- a models.py file and updated requirements.txt to include sqlalchemy 2.0

First try?

- it worked on the first try.

2.

Prompt

I have an existing ProcureGIX Streamlit app. I need to add multi-item submission to the student order form.

What to add:

- Below the existing item fields, add an "Add Another Item" button that appends a new set of item fields (Item Name, Quantity, Price, Link to Purchase, Notes)
- Students can add as many items as they need before submitting
- All items in one submission are grouped under the same Team # and CFO
- A "Remove" button next to each added item row so students can delete a row if they made a mistake
- On submit, all items are saved together as one order group in the database

Do not change the Team #, CFO Name, Provider, Class selector fields or the admin dashboard.

Result

- expanded the student form, allowing user to submit multiple orders
- Worked on the first try

3.

Prompt

- I have an existing ProcureGIX Streamlit app. I need to add a student-facing dashboard as the default view when a student logs in.
- What to add:
- When a student selects the Student role, show a dashboard as the home screen instead of going straight to the form
- At the top of the dashboard, show a Budget Summary section with one card per class the student is enrolled in, displaying:
- Class name
- Total budget for the class
- Amount spent so far (sum of all approved orders)
- Amount remaining
- A simple progress bar showing budget utilization
- Below the budget summary, display a table of all orders submitted by that student (filtered by CFO name) with the following columns: Item Name, Quantity, Price, Provider, Class, Status, Date Submitted
- Status should be color-coded: Pending (yellow), Approved (green), Rejected (red), Ordered (blue), Arrived (grey)
- A prominent "Submit New Order" button at the top of the dashboard that navigates to the existing submission form
- After submitting a new order, return the student back to their dashboard

- Admin side: Dorothy should be able to set the budget per class from her admin dashboard. This value is what populates the student-facing budget cards.
- Do not change the submission form itself or the existing admin order table.
-

Result

- Worked on the first time, creates the base home page code for students and their current classes and spendings. Give admin ability to set budget for classes

Accessibility baseline results: Briefly report:

- color contrast check result (pass/fail and any fix)
    - Foreground: #31333f
    - Background: #FFFFFF
    - Contrast ratio: 12.53:1
- semantic headings check result (pass/fail and any fix)
    - passed

Reflections

1. What surprised you about AI-assisted coding? Was it easier or harder than you expected? What moment made you think "wow" or "wait, that is not right"?
    a. I think AI-assisted coding has its pros and cons. It feels not as surgical compared to manually coding. For instance, if you want it to create a certain UI, it can give you the very rough base for the design, and then you have to go through with your own coding for a more precise tune up of certain details. For this project specifically, I realized Streamlit is not the most ideal for making very distinct UI designs, and would've preferred to use React with Tailwind where there are more numerous UI libraries available for customization.

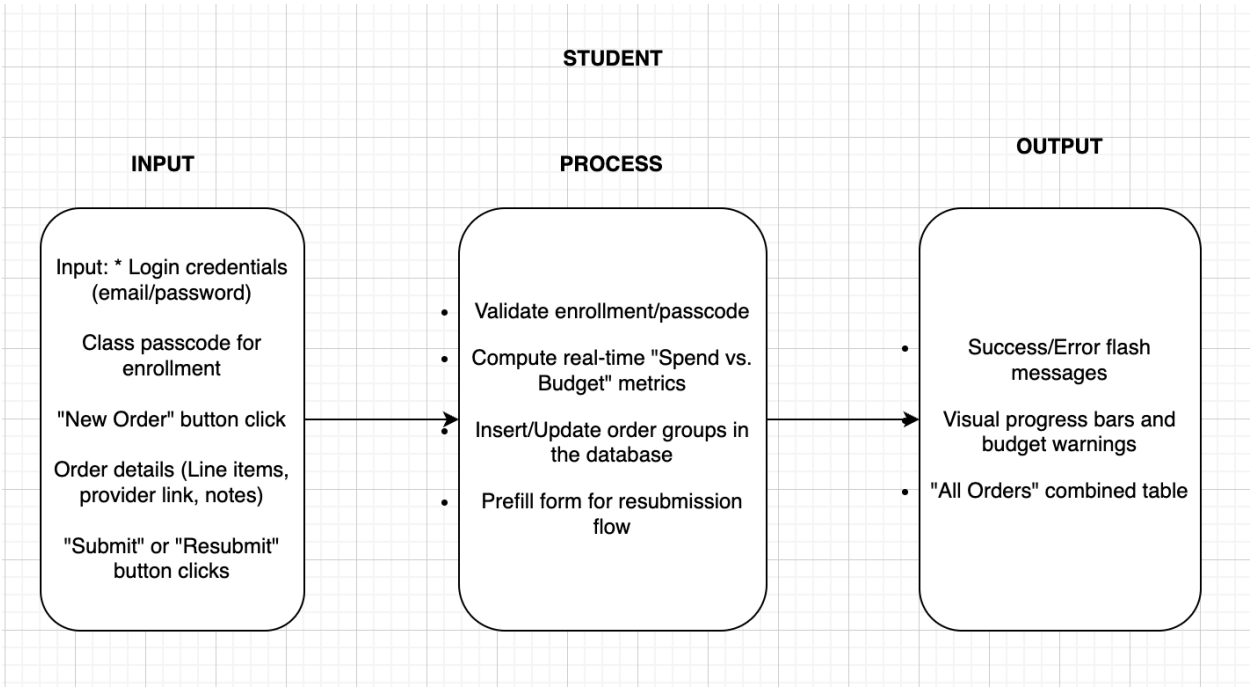
2. What did the AI get wrong? Did you encounter errors, unexpected behavior, or code that did not match what you asked for? How did you fix it?
    a. For the most part, coding now is being able to be specific enough with your request so that the AI can understand. There were many times when AI started to hallucinate and started to reference code or features that did not exist. Fixing it involved breaking up tasks into separate agents to keep the features localized and avoid piling up on context usage.

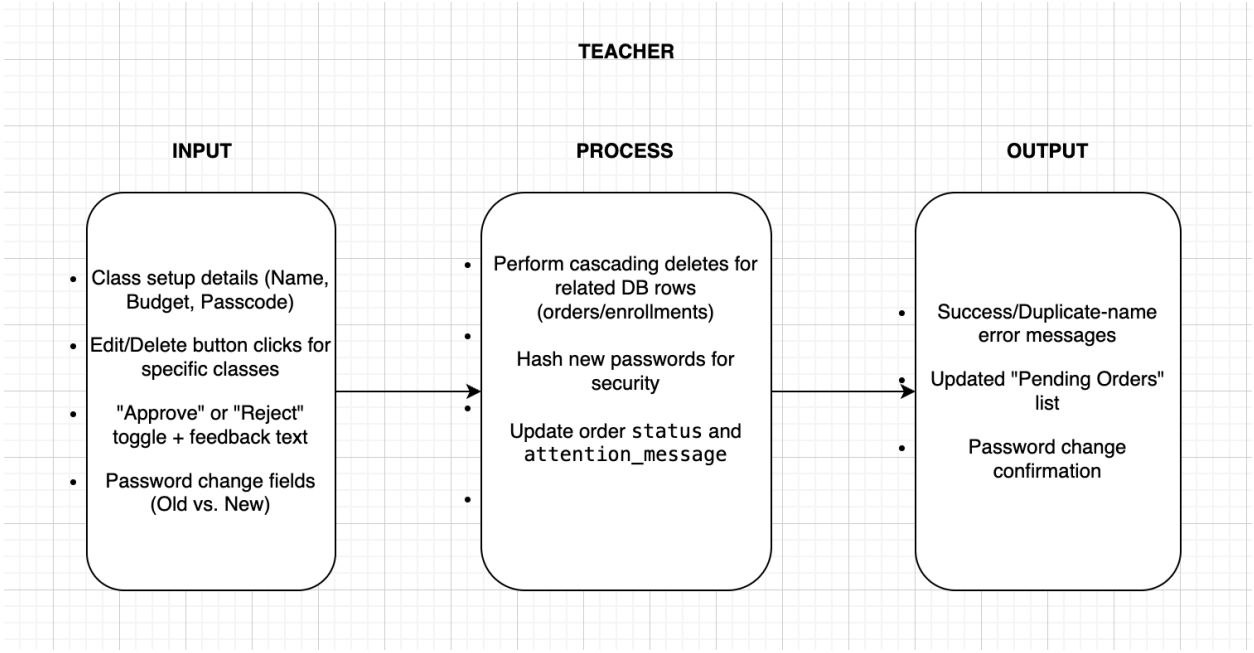
3. Could you explain your code? Pick one section of your app -- could you walk a classmate through what it does, line by line, without looking at Cursor's explanation?
    a. I would be able to explain the purpose of each of the files, however I would have trouble explaining exactly what each line of code does especially since it involves syntax from an unfamiliar framework that would require in depth research or prompting cursor to explain.

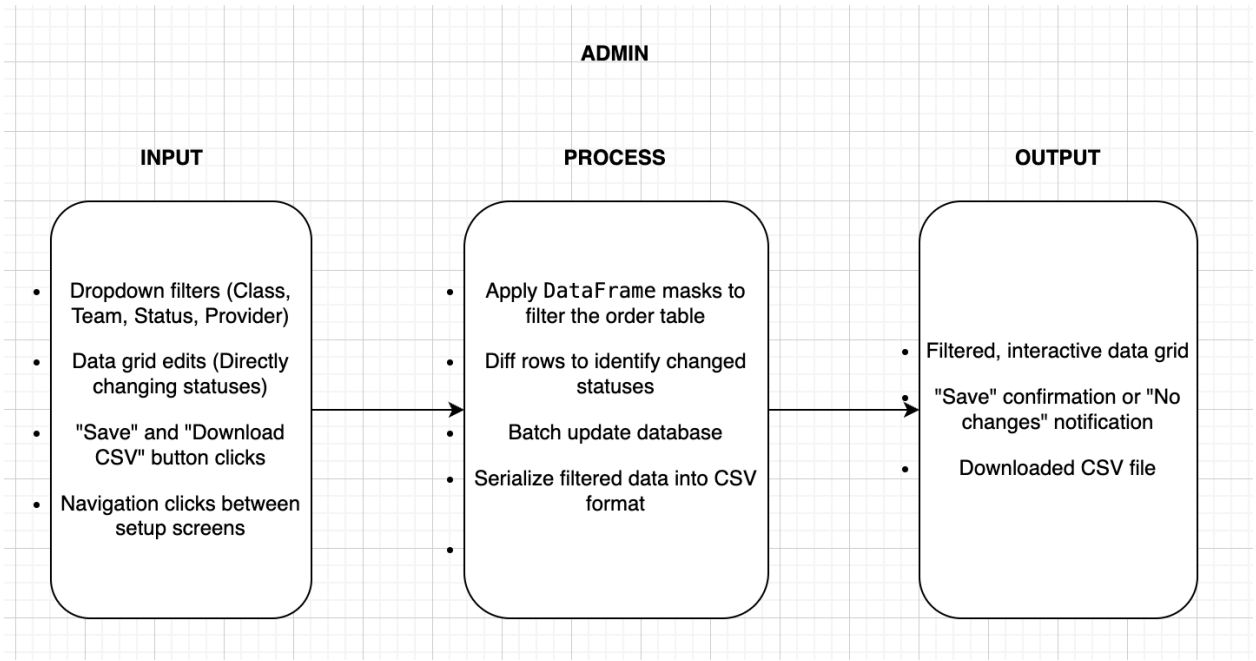
4. What did you learn from the interview? How did talking to Dorothy (Program Coordinator, Student Purchasing) change what you built, compared to what you might have built without that conversation?
    a. I think there is a lot that goes on, under the hood that as someone who is only involved in one side of the process (purchasing/filling out excel form) doesn't get to see. I think if Dorothy did not walk us through the whole process, my solution would be localized in the portion that I am most familiar with, which is the student facing part of the application.

Component C

1. Architecture diagram







2. Design Decision Log

Respond to the Decision Prompt

"Why did you put everything in one file (app.py) instead of splitting it into multiple files?"

- When Cursor was building the app, it put all of the code into 1 app.py file and a db.py file, however as app.py got to around 1500+ lines of code, I asked cursor to refactor the code. I decided to split it because I believe it would be more

efficient for me to @ a specific file when changing or adding a feature, so that way Cursor doesn't have to sift through the entire codebase for a specific section. This way, Cursor also doesn't accidentally alter sections of the app that I want to keep the same. In the future if I needed to add more features, I would add them to the specific scripts, or create new ones if appropriate.

Component D

- Validation results (the completed smoke-test table)

#	Feature Tested	Action Taken	Expected Result	Actual Result	Pass/ Fail
1	App loads	"streamlit run app.py" in terminal	login screen shown	login screen shown	Pass
2	Student login (success)	Student → valid email + password → Log in	Dashboard loads; header shows team / CFO / email, current classes and notification section	Shows student dashboard, current classes/enroll, notification, name, logout button	Pass
3	Student login (failure)	Student → wrong password → Log in	"Invalid email or password"; stay on sign-in	Invalid email or password, or the address is not an @uw.edu account. stays on login screen	Pass

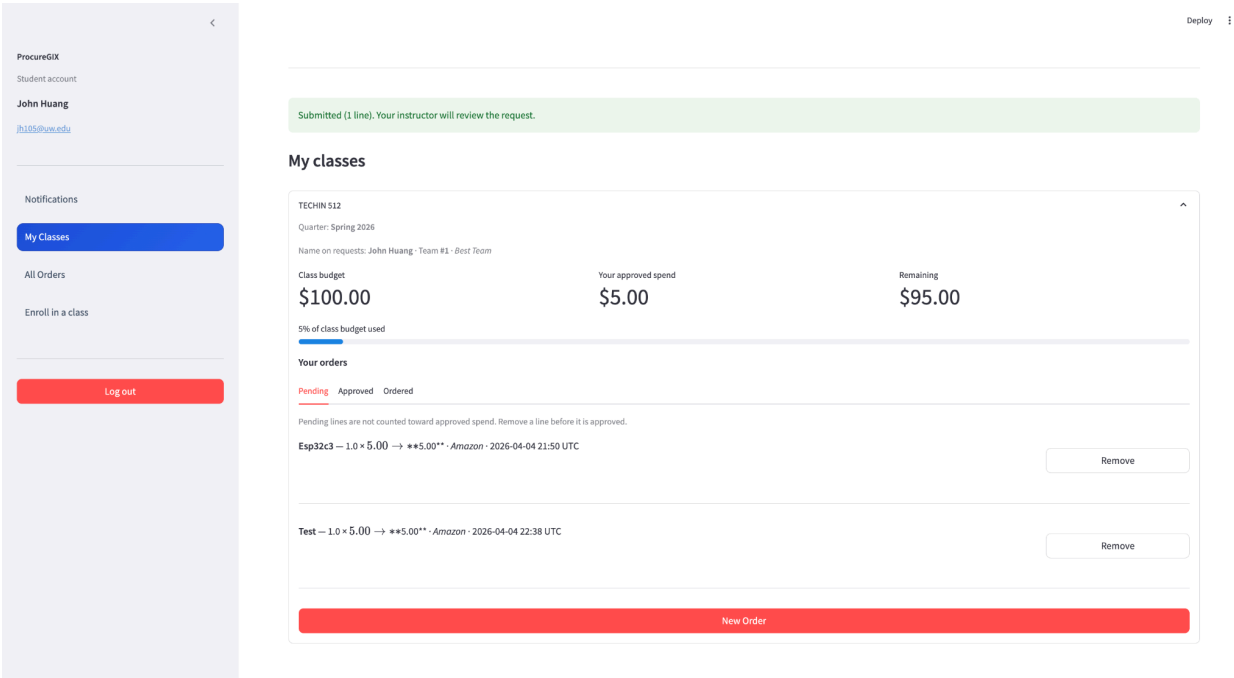
4	Student logout	Signed in → Log out	Back to student sign-in	back to student sign in	Pass
5	Enroll in class (success)	Enroll UI → pick class + correct passcode → Enroll	Success message; class under *My classes*	Success message, class card appears under My classes	Pass
6	Enroll (wrong passcode)	Enroll with bad passcode	Error; not enrolled	Incorrect enrollment passcode, not enrolled	Pass
7	Submit order	New Order → fill required fields → Submit order → Confirm	Success flash; row(s) in *Your orders* / *All orders*	Success message, row added to class order and all orders	Pass
8	Order validation	Submit with missing item or link or qty	Errors shown; no order created	Error shown, no orders created	Pass
9	Budget enforcement	Submit order that would	Error; order not placed	Error message saying exceeding budget, no order placed	Pass

		exceed class budget			
10	Notifications + resubmit	Rejected line exists → Notifications → Resubmit	Opens order form for follow-up		
11	Teacher login	Teacher → valid credentials → Log in	Teacher dashboard loads	Teacher dashboard loads	Pass
12	Teacher create class	Add New Class → name, budget, passcode → Create class	Class appears; students can enroll with passcode	Creates class, appears in teacher dashboard, students can select and enter passcode to join	Pass
13	Approve pending order	Pending line → Approve	Status approved; disappears from pending	Status approved, students can no longer cancel, disappears from pending	Pass
14	Reject with feedback	Reject → enter feedback →	Status rejected; student can see follow-up	Status rejected, students can see reason why, disappears from pending	Pass

		Confirm rejection			
15	Reject blocks empty feedback	Confirm rejection with empty feedback	Error; stays in reject flow	Error message, stays on same page	Pass
16	Admin: view orders	Admin (default orders view)	Order table visible when data exists	Order table visible when orders placed	Pass
17	Admin: filters	Change filter multiselects (e.g. status/class)	Rows match filters only	rows correctly match filtered parameters	Pass
18	Admin: save status	Edit status in grid → Save status changes	Success; persists after refresh	success message, persists after refreshing	Pass
19	Admin: export CSV	Download filtered data as CSV	File downloads with expected columns	File successfully downloaded to CSV	Pass

20	Enrolling into same class	Student try to enroll into same class	Deny, you are already enrolled in this class	Deny, you are already enrolled in this class	Pass

- Screenshot of your running app showing at least one tested feature

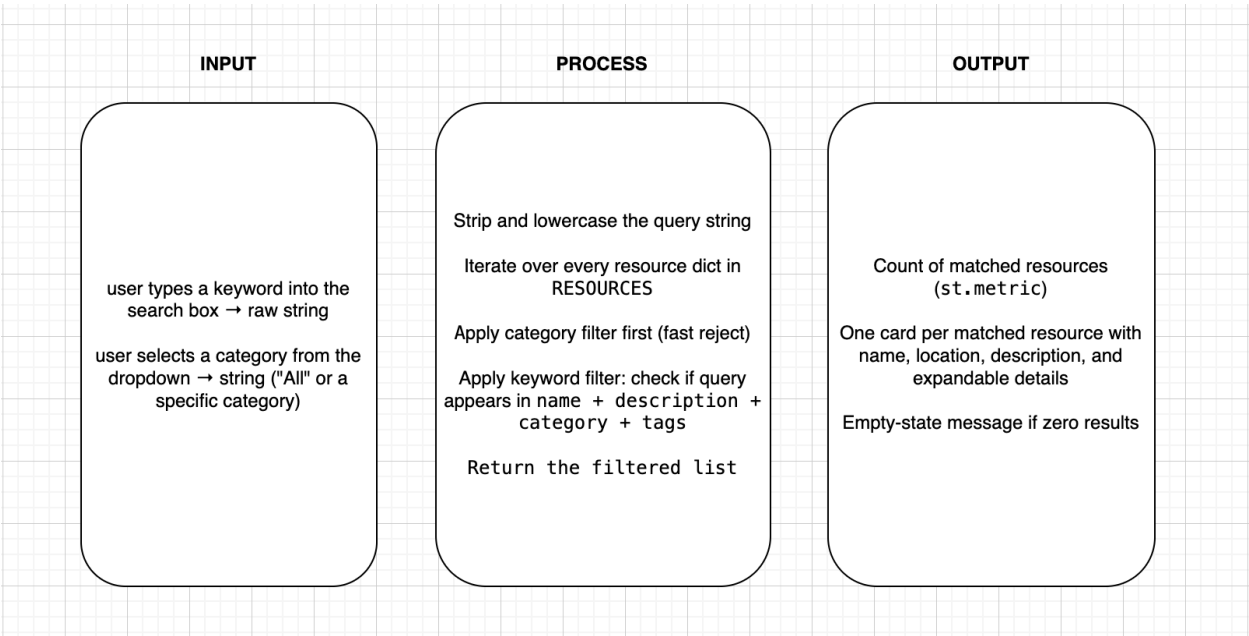


- Success Message after purchasing an item

- Quality gate checklist (completed)
  a. Smoke test table completed with 3 tested features (Done)
  b. Any failed test is either fixed and re-tested, or clearly documented (Done)
  c. Screenshot of the running app included (Done)
  d. Accessibility baseline results recorded (color contrast + semantic headings) (Done)

Component E

1. Wayfinder app implementation in your repo (code and data structure)
2. I/P/O diagram for the Wayfinder app



3. Edge-case validation notes for 2 edge cases (with why they matter)
    a. Edge Case 1: Whitespace-only query
        i. searching with white space only should return all results instead of nothing. By using strip function we can remove the white space and have it return all results instead of no results.
    b. Edge Case 2: mixed capitalization
        i. search uses exact matches, if we dont normalize case sensitivity of the string, no results will show up if there is a slight case mismatch which is not the intended function
4. One `assert` statement for data integrity (include it in code and mention where)
5. Prompt log excerpt: initial prompt + one refinement, with a brief note on what changed and why

Initial:

- "Build a Streamlit app that shows GIX campus resources with search and filters"
- **Problem**: too vague, no defined data structure, UI got mixed into it as well.

Revised:

- Create a Streamlit app for a GIX campus resource finder. Define a Python list of dicts. Each dict should have these fields: name (str), category (str), location (str), hours (str), access (str), description (str), tags (list of str), cost (str), and contact

(str or None). Populate it with at least 8 real-sounding GIX campus resources placeholders such as: makerspace, bike storage, free printing, quiet study room, collaborative studio, graduate lounge, IT help desk, and career services. Don't add any UI yet, just the data and an assert statement that checks every resource has all required keys.

- Result: splitting up parts of the projects into different prompts allows the AI to focus on certain sections and allows us to debug a smaller subsection of code when errors occur.